

Git

Git初识

Git是什么

Git是一个分布式版本控制系统，简单来说就是一个可以记录文件变化，支持多人协作的工具。

Git的作用

记录项目历史；多人协作；代码备份与共享。

为什么要用Git

高效处理大小项目；灵活的分支管理；离线工作。

Git的优点

1.分布式架构：每个开发者都有完整的仓库，支持离线工作和独立开发 2.速度快：大多数操作在本地完成，相应迅速 3.强大的分支和合并功能 4.开源免费

Git的缺点

1.学习曲线较陡：对于新手来说，概念和命令较多，需要时间掌握 2.不适合超大文件 3.潜在的安全风险：私有项目若误设为公开，可能导致代码泄露

Git实战

- 首先请新建一个分支 A，本题代码不与其他题目代码提交在同一个分支下
- 产品经理希望你使用 C 语言写一个猜拳小游戏，并将其命名为 `finger-guessing.c`，赢一次以后即退出游戏。完成后，请将其推送到远程仓库（注意远程仓库应当也提交到分支 A）。
- 产品经理觉得这个游戏过于简单，他希望增加一个计时功能，限时10秒内获得胜利，否则直接退出游戏。完成后，请将其推送到远程仓库。
- 倒霉的产品经理在第999次失败后愤怒地要求取消计时功能，可是我们的代码已经改得一团糟了，难道我们要从头开始检查吗？并不。请通过 `git` 版本控制回退到添加计时功能以前的版本。完成后，请将其推送到远程仓库。
- 一次就退出太短了，产品经理希望增加到五局三胜制，平局不记录，请注意输出胜局情况。完成后，请将其推送到远程仓库。
- 输了很多次的产品经理希望拥有一个作弊功能，如果输入 `zzz` 则进入每把必赢的作弊模式。但是，鬼知道产品经理还会提出多少功能，所以请新建一个分支 B，在 B 分支中添加该功能，并将修改后的文件推送到远程仓库（注意远程仓库此时应提交到分支 B）。
- 产品经理对五局三胜做出一些修改，请转到分支 A，将五局三胜改为三局两胜，并将修改后的文件推送到远程仓库（注意远程仓库此时应提交到分支 A）。
- 产品经理终于满意了，她决定上线这款产品。请合并 A 和 B，同时保留三局两胜制和作弊模式两个功能。完成后，请将其推送到远程仓库。

C语言

一、ASCII码

- 请在VSCode下运行以下代码，并给出输出结果。

```
#include <stdio.h> int main(){ char a = 'a'; printf("%d",a); return 0; }
```

输出结果为“97”

原因：在 C 语言中，char 类型在内存中存储的是一个整数值，这个整数值通常是该字符在 ASCII 码表中对应的编码，在标准的 ASCII 码表中，小写字母 'a' 对应的十进制整数值是 97，printf 是一个格式化输出函数，第一个参数 "%d" 是一个格式说明符，它告诉 printf 函数应该将第二个参数 a 按照十进制整数 (decimal integer) 的格式来打印，因此，printf 不会打印出字符 'a' 本身，而是打印出变量 a 在内存中存储的整数值，也就是 97。

区别

- '1'是一个文本符号或图形,1是一个数学值或数量，用来进行计算。
- '1'是char (字符类型), 1是整型
- '1'存储的是它在 ASCII 码表中的编码值，即整数 49；1存储的是它的二进制值，即 00000001 (以8位为例)。
- '1'用于文本处理、显示、字符串操作等；1用于数学运算、计数、循环、数组索引等。

二、数组

- 请完成以下的C语言程序，实现输入5个学生的成绩，然后计算并输出平均分

```
//预期输出结果示例: 请输入5个学生的成绩: 请输入第1个学生的成绩: 85 请输入第2个学生的成绩: 92 请输入第3个学生的成绩: 78 请输入第4个学生的成绩: 65 请输入第5个学生的成绩: 90 5个学生的平均分是: 82.00
```

- 请修改之前的平均分计算程序，让用户可以选择输入的学生数量（而不是固定的5个）

```
//预期输出结果示例: 请输入学生人数: 6 请输入6个学生的成绩: 请输入第1个学生的成绩: 85 请输入第2个学生的成绩: 92 请输入第3个学生的成绩: 78 请输入第4个学生的成绩: 65 请输入第5个学生的成绩: 90 请输入第6个学生的成绩: 82 6个学生的平均分是: 82.00
```

- 请继续修改程序，增加一个功能：显示每个分数有多少人获得。

```
//预期输出结果示例: 请输入学生人数: 5 请输入5个学生的成绩: 请输入5个学生的成绩(0-100分): 请输入第1个学生的成绩: 85 请输入第2个学生的成绩: 92 请输入第3个学生的成绩: 85 请输入第4个学生的成绩: 78 请输入第5个学生的成绩: 85 5个学生的平均分是: 85.00 分数分布统计: 分数 人数 ===== 78 1 85 3 92 1
```

三、循环结构

- 请使用for循环计算一个正整数的阶乘。阶乘的定义为： $n! = 1 \times 2 \times 3 \times \dots \times n$ ，且 $0! = 1$ 。（阶乘结果数据类型请使用long long）

```
//预期输出结果示例 请输入一个非负整数: 5 5! = 120
```

- 请使用while循环编写一个程序，计算并输出斐波那契数列的前n项（n由用户输入）。斐波那契数列定义为： $F(0) = 0$ $F(1) = 1$ $F(n) = F(n-1) + F(n-2)$ ($n \geq 2$)

```
//预期输出结果示例 请输入打印项数:5 斐波那契数列前5项为:0 1 1 2 3
```

- 作为了解，请自行上网查询资料学习函数以及递归函数，并用递归函数再次完成计算阶乘的程序。

四、指针

- 指针基础概念

```
#include <stdio.h> int main() { int num = 10; int *ptr = ____; // 填空：将ptr指向num printf("num的值: %d\n", ____); // 填空：通过指针访问num的值 printf("num的地址: %p\n", ____); // 填空：通过指针获取num的地址 *ptr = 20; // 通过指针修改num的值 printf("修改后num的值: %d\n", num); return 0; }
```

```
&num *ptr ptr
```

- 指针与数组的关系

```
#include <stdio.h> int main() { int arr[5] = {1, 2, 3, 4, 5}; int *ptr = arr; // 数组名就是数组首元素的地址 // 使用指针访问数组元素 for(int i = 0; i < 5; i++) { printf("arr[%d] = %d", i, ____); // 填空：通过指针访问数组元素 printf(", 地址: %p\n", ____); // 填空：获取每个元素的地址 ptr++; // 指针移动到下一个元素 } return 0; }
```

```
*ptr ptr
```

- 指针的运算

```
#include <stdio.h> int main() { int arr[3] = {10, 20, 30}; int *ptr = arr; printf("%d\n", *ptr); // 输出: ? printf("%d\n", *(ptr+1)); // 输出: ? printf("%d\n", *ptr+1); // 输出: ? ptr++; printf("%d\n", *ptr); // 输出: ? return 0; }
```

```
10 20 11 20
```

- 编写一个函数swap,用于交换两个整数的值